P3374R1
Adding formatter for fpos<mbstate_t>

Published Proposal, 2024-12-06

This version:
https://extra-creativity.github.io/public/cpp/proposals/add%20formatter%20for%20fpos-R1.html

Author:
Liang Jiaming (Peking University)

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper aims to add formatter for `std::fpos<std::mbstate_t>` widely used by position indicator APIs in stream to make it more convenient and robust to output.

Table of Contents

§ 1. Introduction

Stream-based I/O is designed since C++98 to give a thorough abstraction to I/O. An important part of the abstraction is position indicator, which is used to tell and seek the current position. Such position is most commonly conveyed by `char_traits<T>::pos_type`, which is `fpos<typename char_traits<T>::state_type>` by default and the only instantiation used in the standard library is `fpos<mbstate_t>`. It's widely implemented to be able to be output by stream `operator<<`; however, it cannot be output directly by `print` in C++23 astonishingly due to lack of `formatter`. Moreover, the state of the position is usually neglected, making it not robust in some cases to be output by `operator<<`. This paper aims to add a specialization for `fpos<mbstate_t>` to solve these problems.

§ 2. Revision History

Changes since R0

These changes are mainly suggested by SG16, as discussed and polled in the regular meeting.

- Format the state as Boolean instead of a recoverable descriptor.
- Fix several minor wording problems.

§ 3. Motivation

Let's see tony table to illustrate it directly:

Before

```
std::ofstream s{"some_file"};
// Do some output...
std::cout << s.tellp();        // ❓Yes on almost all implementations, but not robust
std::println("{}", s.tellp()); // ❌Compile error...
// To make it work same as operator<<, we have to write...
std::println("{}", (std::streamoff)s.tellp());    // 😢What?
std::println("{}", s.tellp() - std::streampos{}); // 😢No way...
```

After

```
std::ofstream s{"some_file"};
// Do some output...
std::cout << s.tellp();           // ❓Yes on almost all implementations, but not robust
std::println("{}", s.tellp());    // ✅Yes and more robust!
std::println("{:d}", s.tellp());  // ✅Non-robust way can be controlled by users explicitly.
std::stringstream s2{"ABC"};
std::println("{:d}", s2.tellp()); // ✅Especially for streams that don't need codecvt.
```

§ 4. Design Decision

§ 4.1. Core Problem

§ 4.1.1. Not an integer, only convertible

For C programmers and those who don't take a thorough look at the design of stream, they're likely to regard the position as an integer directly since `ftell` just returns so. Actually, in C++ it's designed as follows:

```
template<typename CharT, typename CharTraits = char_traits<CharT>>
class basic_iostream
{
public:
    using pos_type = typename CharTraits::pos_type;
    pos_type tellg();
    pos_type tellp();
};
```

And the most commonly used types are alias like:

```
template<typename CharT, typename CharTraits = char_traits<CharT>>
class basic_fstream : public basic_iostream<CharT, CharTraits> { ... };

using fstream = basic_fstream<char>;
using wfstream = basic_fstream<wchar_t>;
```

So the return type of `tellg/p` is usually determined by `char_traits<CharT>::pos_type`, where `CharT` is `char` or `wchar_t`. They're defined as `fpos<typename char_traits<CharT>::state_type>`, and the only used instantiation in the standard library is `fpos<mbstate_t>`. `fpos` only supports limited integer operations, like subtracting another `fpos` to get `streamoff`, or adding a `streamoff` offset to get a new `fpos`. Particularly, `streamoff` is regulated to be an alias of a signed integer type, and `fpos` should be convertible to `streamoff` to make expression `streamoff(pos)` compile (See [stream.types] and [fpos.operations] in the standard).

Though such requirement can be implemented as follows:

```
template<typename StateT>
class fpos
{
    explicit operator streamoff() const { ... }
};
```

As explicit conversion operator is only supported since C++11 while streams are introduced in C++98, the existing mainstream implementations, like MS-STL, libstdc++, libc++, and many other older implementations like Apache stdcxx, STLport (stlport/stl/char_traits#92), all choose to make it not `explicit`, even though it could be strengthened since C++11.

Such phenomenon make an illusion to C++ programmers that "it's just an integer". Specifically, for output, the `operator<<` has all overloads like:

```
basic_ostream& operator<<(int);
basic_ostream& operator<<(long);
basic_ostream& operator<<(long long);
```

This enables implicit conversion from `fpos` to `streamoff` to match one of the overloads and makes output successful. However, it fails to work when it comes to `format`, since template doesn't try to do implicit conversions in such cases. The `formatter` specialization for `int`, `long` and `long long` cannot be utilized by `fpos` without explicit conversion.

§ 4.1.2. Not only the integer, though convertible

Another problem that's usually neglected is that `fpos` doesn't merely contain the position integer; it also has a `state` as conveyed by the template parameter, most typically `mbstate_t`. It's used to determine the current state of character conversion, like for `codecvt` in locale. By default, `fpos` assigned by a `streamoff` will have a value-initialized state, which means the initial state. However, sometimes it's possibly not "initial", so "output it as an integer -> input it to an integer -> assign it back to `fpos`" is both unsafe and incorrect.

For instance, though rarely happen, if some derived class of `basic_streambuf` allows partial conversion when overriding `overflow` (since it's only regulated to prepare space for at least one `CharT`), `tellp` of its stream may also report a `fpos` with partial status. Anyway, it's incomplete to only report the position integer in some cases.

§ 4.2. Proposed Solution

To make it both safe and convenient, we propose to add formatter specialization for `fpos<mbstate_t>`. Considering that almost all behaviors of `mbstate_t` are implementation-defined, it seems meaningless to regulate its format specifications. It's also suggested by SG16 to not provide an implementation-defined descriptor to enable possible scanner ([P1729]) to restore it. Thus, we just propose to additionally output whether it's in the initial state.

So to be specific, the formatter specialization of `fpos<mbstate_t>` should behave as follows:

- When no specification is given (i.e. `{}` or `{:}`), `format` should produce `"(position, mbsinit(&state))"`, where the latter is either `true` or `false`;
- When some specifications are given (e.g. `{:d}`), only the position will be output in the way determined by the format specifications.

§ 4.3. Possible Future Evolution

There is a related issue that is considered but not proposed in this paper. That is, it could be discussed whether we need to change the behavior of `operator<<`, like making conversion operator of `fpos` to `streamoff` explicit or overloading `operator<<` for `fpos<mbstate_t>`. Such breaking change may or may not be expected by many. It may be solved, if necessary, in other future proposals.

§ 5. Standard Wording

We propose to add wording in [fpos.operations]:

2.Stream operations that return a value of type traits::pos_type return P(O(-1)) as an invalid value to signal an error. If this value is used as an argument to any istream, ostream, or streambuf member that accepts a value of type traits::pos_type then the behavior of that function is undefined.

3. The formatter of `fpos<mbstate_t>` should behave as follows:

```
namespace std {
 template<class charT>
 struct formatter<fpos<mbstate_t>, charT> {
 private:
  formatter<streamoff, charT> underlying_;    // exposition only

  bool need_state_ = false;     // exposition only

 public:
  template<class ParseContext>
   constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);

  template<class FormatContext>
   typename FormatContext::iterator
    format(const fpos<mbstate_t>& ref, FormatContext& ctx) const;
 };
}

template<class ParseContext>
 constexpr typename ParseContext::iterator
  parse(ParseContext& ctx);
```

Effects: Sets `need_state_` to `true` if *format-specifier* or *format-spec* ([format.string.general]) is not present, otherwise same as `underlying_.format(ctx)`.

Returns: An iterator past the end of *format-spec*.

```
template<class FormatContext>
 typename FormatContext::iterator
  format(const fpos<mbstate_t>& ref, FormatContext& ctx) const;
```

Effects: Writes the following into `ctx.out()`:

- — If `need_state_` is `false`, then as if `underlying_.format(static_cast<streamoff>(ref), ctx)`;
- — Otherwise,
  - — *STATICALLY-WIDEN*`<charT>("(")`,
  - — the result of formatting `static_cast<streamoff>(ref)` via `underlying_`,
  - — *STATICALLY-WIDEN*`<charT>(", ")`,
  - — the result of formatting the bool value to denote whether the `ref.state()` is in the initial state, which is same as calling `mbsinit`([c.mb.wcs]),
  - — *STATICALLY-WIDEN*`<charT>(")")`.

Returns: An iterator past the end of the output range.

§ 6. Impact on Existing Code

This is a pure extension to the standard library so there won't be severe conflicts with the existing code. The only possible conflict is that some existing code has already added specialization on `fpos<mbstate_t>`, but it seems that no open-source code on Github does so.

§ 7. Implementation

Since many other formatters have already require widen characters, we just use `WIDEN` to represent it in code below for simplicity. Inheritance may be used to implement it:

```
template<typename CharT>
struct formatter<fpos<mbstate_t>, CharT> : formatter<streamoff, CharT>
{
private:
    using Base = formatter<streamoff, CharT>;
    bool need_state_ = false;

public:
    constexpr auto parse(const auto& ctx)
    {
        auto it = ctx.begin();
        if (it == context.end() || *it == WIDEN('}'))
        {
            need_state_ = true;
            return it;
        }

        return Base::parse(ctx);
    }

    auto format(const fpos<mbstate_t>& value, auto& ctx) const
    {
        if (need_state_) {
            auto state = value.state();
            return format_to(ctx.out(), ctx.locale(), WIDEN("({}, {})"),
                             static_cast<streamoff>(value), mbsinit(&state));
        }
        return Base::format(static_cast<streamoff>(value), ctx);
    }
};
```

We notice that:

— What `.state()` returns is the value type, which may introduce unnecessary copy in `auto state = value.state()` since the state may be stored directly in `fpos<mbstate_t>`. If the compiler is unable to optimize it out, unexposed members of `fpos` may be utilized (e.g. by declaring the formatter as friend) to write it like `mbsinit(&value.inner_state)`.

— MS-STL and libstdc++ can still utilize the implicit conversion so the `static_cast` in `Base::format` can be omitted. libc++ checks whether `value` is integer in the template `Base::format` method and thus implicit conversion cannot help.

§ 8. Acknowledgement

Thanks to Victor Zverovich, the author of [fmt], and Arthur O’Dwyer for rounds of suggestions and discussions on generalization of the conversion and encouragement to post this paper when the idea is first proposed. Thanks to Tom Honermann for advice on formatting the state type and assistance in D3374R0. Thanks to all members in SG16 for discussion in the regular meeting and mailing list in P3374R0. I’d also like to extend my gratitude to Peking University for giving me a colorful undergraduate life in the last four years.

§ References

§ Informative References

[FMT]
Victor Zverovich; et al.. *The {fmt} library*. URL: https://github.com/fmtlib/fmt

[P1729]
Elias Kosunen, Victor Zverovich. *Text Parsing*. URL: https://wg21.link/p1729